CLIENT BRIEFING

AI Agent Security Architectures

> Five ways to deploy an AI agent, each shown as a diagram of security zones with the risks at every crossing and the safeguards that address them.

THE IDEA IN ONE PARAGRAPH

Treat an AI agent like a new employee, not like a piece of software. It has a brain (the AI model), a job description (its instructions), a set of tools, an identity badge, and a manager who limits what it may do. The AI model only decides what to do next. Everything it actually does goes through software that you control, and that is where security is enforced. The safeguards in this document never rely on the model behaving well.

HOW TO READ EACH DIAGRAM

Each box is a zone. Boxes inside boxes are zones within zones. Every arrow is a request crossing from one zone to another, and each crossing is a place to check identity and permission. Green tags mark where a safeguard sits; the table after each diagram explains every tag. The same example is used throughout: an HR assistant that reads leave balances and approves leave.

[ ] Outside your control Cloud account Your data centre B3 Trust boundary C5 Control (explained in the table that follows) Arrow: a request crossing from one zone to the next

TERMS USED IN THIS DOCUMENT

TERM	WHAT IT MEANS
Agent	An AI model plus the software around it that lets it use tools and take actions. The model decides; the software acts.
Trust boundary	A line where one zone stops trusting another. Anything crossing it must prove who it is and what it is allowed to do. Shown as B1, B2 and so on.
Control	A specific safeguard placed at a boundary to reduce a risk. Shown as C1, C2 and so on.
Token	A short-lived digital pass. It says who you are and what you may do, and it expires quickly so a stolen one is of little use.
Least privilege	Giving the agent only the access its job needs, and nothing more.
On behalf of	The agent acts with the employee’s own permissions, so it can never see more than that employee could see directly.
Tool gateway	A single checkpoint that every tool request must pass through. Rules are enforced here, outside the AI model.
Guardrail	A filter that screens text going into and out of the model for attacks, personal data and unsafe content.
Prompt injection	Hidden instructions planted in a message or document to trick the model into doing something harmful.
MCP	Model Context Protocol. A standard plug that lets agents connect to tools in a consistent way.

The five architectures at a glance

ARCHITECTURE	WHO RUNS THE PLATFORM	DOES DATA LEAVE YOUR PREMISES?	MAIN RISK TO MANAGE	SUITS
1. AWS Bedrock	AWS	Yes, into your own AWS account	Giving the agent too much access; misuse of tools	Teams already on AWS that want to move quickly
2. Azure Foundry	Microsoft	Yes, into your own Azure subscription	Too many agent identities with unreviewed access	Organisations built around Microsoft 365
3. Self-hosted	You	No	Building and maintaining every safeguard yourself; trusting model files	Regulated sectors with strict data-residency rules
4. Hybrid with AWS	You run the agent, AWS runs the model	Only the cleaned-up question	What the question contains; the cloud credential kept on-premises	Data must stay in-house but a strong cloud model is needed
5. Hybrid with Azure	Microsoft runs the agent, you run the tools	Tool results travel to the cloud	The cloud agent becoming a route into the internal network	A managed agent that must reach older in-house systems

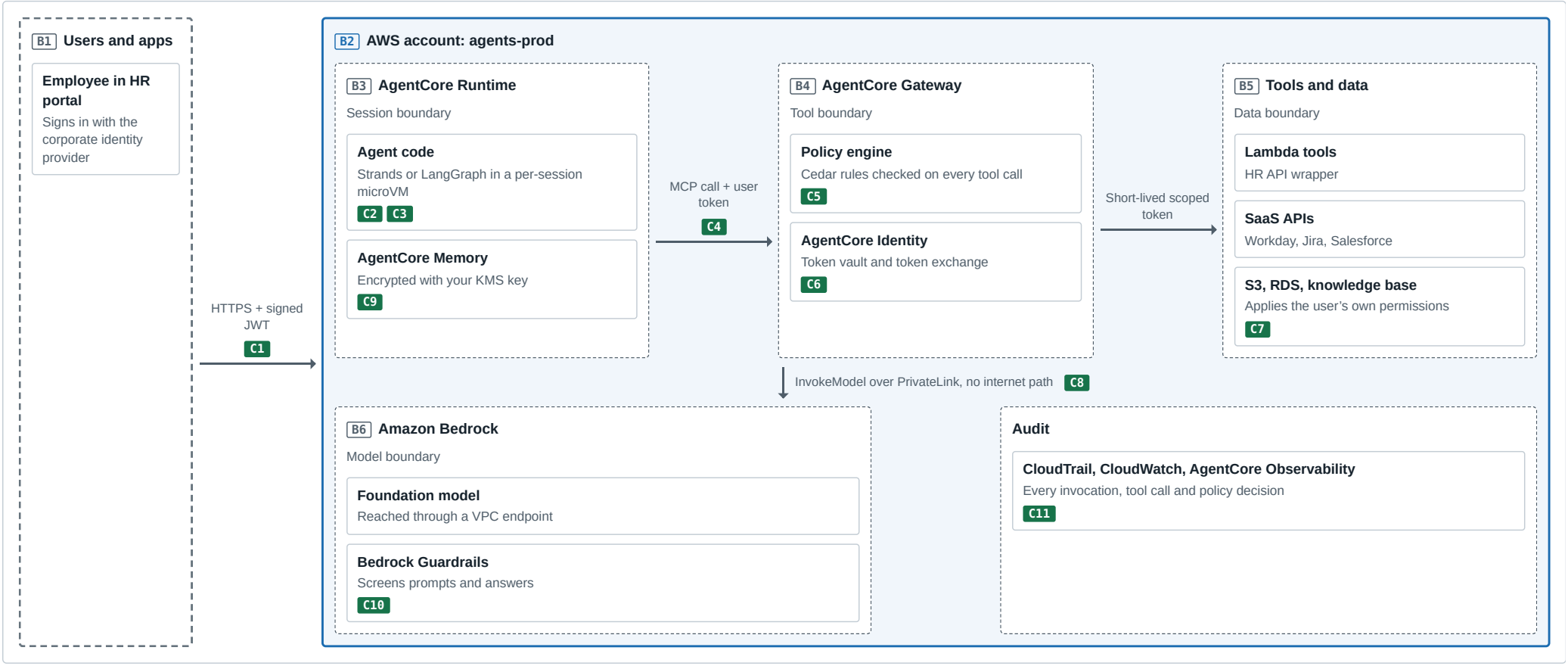
SIX QUESTIONS EVERY DESIGN MUST ANSWER

1. Who is calling the agent? Every user and application must prove its identity.
2. Who is the agent? Each agent needs its own identity, not a shared account.
3. What can the agent reach? Network rules limit where it can connect.
4. What may the agent do? Written rules decide each tool request, outside the AI model.
5. What leaves the environment? Filters and private connections control data going out.
6. Who is watching? Every step is logged so misuse can be detected and investigated.

AWS: Amazon Bedrock AgentCore

> Think of this as renting a fully serviced office from AWS. AWS supplies the building, the locks and the security desk. You decide who gets a badge and which doors each badge opens.

Outside your control · Cloud account · Your data centre · B3 Trust boundary · C5 Control (explained in the table that follows) · Arrow: a request crossing from one zone to the next



WHAT HAPPENS, STEP BY STEP

1. An employee signs in and asks the HR assistant a question.
2. AWS checks the sign-in, then starts a private, sealed session for that one conversation.
3. The AI model reads the question and decides it needs a tool, for example "get leave balance".
4. The tool gateway checks the written rules: is this person allowed to use this tool, with these values?
5. If allowed, the gateway fetches a short-lived pass and calls the HR system as that employee.
6. The answer is screened for personal data and unsafe content, then returned. Every step is logged.

THE BOUNDARIES IN PLAIN WORDS

BOUNDARY	WHAT IT IS
B1 Internet edge	Where people outside meet your system. Nobody is trusted until they prove who they are.
B2 AWS account	Your private space inside AWS. Nothing in it can be reached straight from the internet.
B3 Session	Each conversation runs in its own sealed room, which is wiped when the conversation ends.
B4 Tool gateway	The security desk. Every request to use a tool is checked against written rules before it goes ahead.
B5 Tools and data	Your business systems. They check the employee’s own permissions a second time.
B6 Model	The AI model. Text going in and coming out is screened by a filter.

TECHNICAL SUMMARY

AWS hosts the agent, the tool gateway and the model. You own the identities, the policies and the network rules. The tool gateway is the main enforcement point because its policy runs outside the agent code, so a manipulated model cannot talk its way past it.

RISKS AND THE SAFEGUARDS THAT ADDRESS THEM

TAG	RISK	SAFEGUARD	WHERE IT IS SET
C1	**Anonymous or spoofed caller invokes the agent**	Inbound JWT authorizer pinned to your identity provider, audience and client. A resource policy limits who may invoke the runtime.	AuthorizerConfiguration, runtime resource policy
C2	**One user’s session leaks into another**	Each session runs in its own isolated microVM that is destroyed afterwards. Keep no shared state in agent code.	AgentCore Runtime (built in)
C3	**Agent role can do far more than the job needs**	Execution role allows one model, one guardrail and one gateway. Add a permission boundary and an organisation SCP as a ceiling.	IAM execution role
C4	**Agent reaches tools directly and skips policy**	Tools are only reachable through the gateway. The runtime security group has no other outbound route.	Security group, gateway target list
C5	**Prompt injection makes the agent call a harmful tool**	Default-deny Cedar policies per tool, with conditions on the user’s claims and on the tool arguments. A forbid always beats a permit.	AgentCore Policy engine
C6	**API keys leak through prompts, logs or code**	The agent holds no long-lived secrets. The token vault issues short-lived, per-tool tokens, on behalf of the user where possible.	AgentCore Identity credential providers
C7	**Agent returns data the user is not entitled to**	The downstream system receives the user’s identity and applies its own row and field permissions.	Tool API, Lake Formation, S3 bucket policy
C8	**Data is sent out to the internet**	VPC network mode, PrivateLink endpoints with endpoint policies, no NAT gateway on agent subnets.	NetworkConfiguration, VPC endpoints
C9	**Sensitive or poisoned content persists in memory**	Customer-managed KMS key, a separate memory namespace per user, and a retention limit.	AgentCore Memory settings
C10	**Jailbreak, personal data in answers, harmful output**	Guardrail on input and output: prompt-attack filter, PII masking, denied topics.	Bedrock Guardrails
C11	**Misuse goes unnoticed and cannot be investigated**	Log every invocation and policy decision. Alarm on denied calls and unusual tool volume.	CloudTrail, CloudWatch, GuardDuty

SAMPLE CONFIGURATION

Comments marked C1, C2 and so on show which safeguard each line implements. These samples illustrate the safeguards and are not ready-to-deploy templates.

Runtime: private network and inbound sign-in (CloudFormation)

```
HrAgentRuntime:
  Type: AWS::BedrockAgentCore::Runtime
  Properties:
    AgentRuntimeName: hr_assistant
    RoleArn: !GetAtt HrAgentExecutionRole.Arn          # C3
    AgentRuntimeArtifact:
      ContainerConfiguration:
        ContainerUri: 111122223333.dkr.ecr.ap-south-1.amazonaws.com/hr-agent:1.4.2
    NetworkConfiguration:                               # C8
      NetworkMode: VPC
      NetworkModeConfig:
        Subnets: [subnet-0a1b2c3d, subnet-4e5f6a7b]     # private, no NAT
        SecurityGroups: [sg-0a9e55agent]                # C4: gateway + endpoints only
    AuthorizerConfiguration:                            # C1
      CustomJWTAuthorizer:
        DiscoveryUrl: https://login.example.com/.well-known/openid-configuration
        AllowedAudience: [hr-assistant]
        AllowedClients: [hr-portal-web]
```

Execution role: one model, one guardrail, one gateway (IAM)

```
{
  "Version": "2012-10-17",
  "Statement": [
    { "Sid": "OneModelOnly", "Effect": "Allow",
      "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"],
      "Resource": "arn:aws:bedrock:ap-south-1::foundation-model/MODEL_ID" },
    { "Sid": "OneGuardrailOnly", "Effect": "Allow",
      "Action": "bedrock:ApplyGuardrail",
      "Resource": "arn:aws:bedrock:ap-south-1:111122223333:guardrail/gr-hr01" },
    { "Sid": "OneGatewayOnly", "Effect": "Allow",
      "Action": "bedrock-agentcore:InvokeGateway",
      "Resource": "arn:aws:bedrock-agentcore:ap-south-1:111122223333:gateway/hr-gateway" }
  ]
}
```

Tool policy at the gateway (Cedar)

```
// C5. Any signed-in employee may read a leave balance.
permit(
  principal is AgentCore::OAuthUser,
  action == AgentCore::Action::"HrTarget___get_leave_balance",
  resource == AgentCore::Gateway::"arn:aws:bedrock-agentcore:ap-south-1:111122223333:gateway/hr-gateway"
);

// Only HR staff may approve leave, at most 5 days per call.
permit(
  principal is AgentCore::OAuthUser,
  action == AgentCore::Action::"HrTarget___approve_leave",
  resource == AgentCore::Gateway::"arn:aws:bedrock-agentcore:ap-south-1:111122223333:gateway/hr-gateway"
) when {
  principal.hasTag("department") &&
  principal.getTag("department") == "hr" &&
  context.input.days <= 5
};

// Nobody reaches payroll through this agent, whatever the prompt says.
forbid(
  principal,
  action == AgentCore::Action::"HrTarget___get_salary",
  resource == AgentCore::Gateway::"arn:aws:bedrock-agentcore:ap-south-1:111122223333:gateway/hr-gateway"
);
```

Agent code (Python, Strands)

```
from bedrock_agentcore.runtime import BedrockAgentCoreApp
from strands import Agent
from strands.models import BedrockModel
from strands.tools.mcp import MCPClient
from mcp.client.streamable_http import streamablehttp_client

app = BedrockAgentCoreApp()

model = BedrockModel(
    model_id=MODEL_ID,
    guardrail_id="gr-hr01",        # C10: screens input and output
    guardrail_version="3",
)

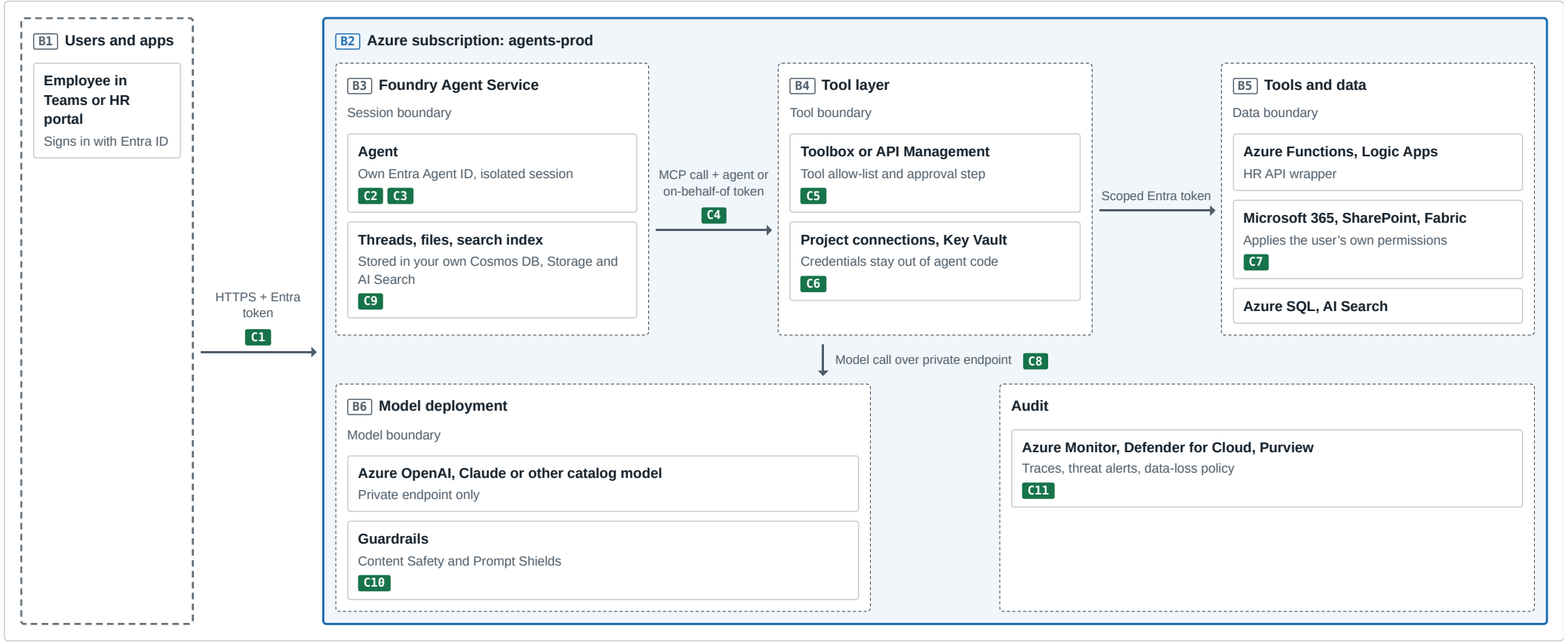
@app.entrypoint
def invoke(payload, context):
    # C6: the user's token travels with the call. No API keys in code.
    user_token = context.request_headers["Authorization"]
    gateway = MCPClient(lambda: streamablehttp_client(
        GATEWAY_URL, headers={"Authorization": user_token}))   # C4
    with gateway:
        agent = Agent(model=model,
                      tools=gateway.list_tools_sync(),
                      system_prompt="You are the HR assistant.")
        return {"result": str(agent(payload["prompt"]))}

app.run()
```

Azure: Microsoft Foundry Agent Service

> This is the same serviced-office idea, run by Microsoft. The main difference is the staff directory: every agent gets its own entry in Entra ID, just like an employee, so you can review it, restrict it and switch it off in one place.

Outside your control · Cloud account · Your data centre · B3 Trust boundary · C5 Control (explained in the table that follows) · Arrow: a request crossing from one zone to the next



WHAT HAPPENS, STEP BY STEP

1. An employee signs in with their normal Microsoft work account.
2. Foundry checks the sign-in and that this person is allowed to use this agent.
3. The AI model decides it needs a tool. Only tools on the approved list are visible to it.
4. For tools that change data, a person is asked to approve the action first.
5. Foundry swaps the employee’s sign-in for a short-lived pass that works for one system only.
6. The answer is screened and returned. Microsoft’s monitoring tools record every step.

THE BOUNDARIES IN PLAIN WORDS

BOUNDARY	WHAT IT IS
B1 Internet edge	Where users meet your system. Only Microsoft work accounts are accepted; shared keys are switched off.
B2 Azure subscription	Your private space inside Azure, closed to the public internet.
B3 Session	Each agent runs in its own isolated session and has its own identity in the staff directory.
B4 Tool layer	The approved list of tools, plus the approval step for sensitive actions.
B5 Tools and data	Microsoft 365 and business systems. They apply the employee’s own permissions and document labels.
B6 Model	The AI model, reachable only on a private connection, with content filters on input and output.

TECHNICAL SUMMARY

Microsoft hosts the agent and the model inside your subscription’s network. Every agent gets its own Entra Agent ID, so it is governed like any other identity: role assignments, access reviews and a single switch to disable it.

RISKS AND THE SAFEGUARDS THAT ADDRESS THEM

TAG	RISK	SAFEGUARD	WHERE IT IS SET
C1	**Anonymous caller or a leaked API key invokes the agent**	API keys are switched off, so only Entra ID sign-in works. A role scoped to one agent decides who may call it. Conditional Access applies to callers.	disableLocalAuth, Azure RBAC
C2	**Agents share one service account, so actions cannot be traced**	Each agent has its own Entra Agent ID. Disable that identity and all of the agent's access stops.	Entra admin centre, Agent ID
C3	**Agent identity has broad rights across the subscription**	Assign the narrowest built-in role at the smallest scope, for example reader on one search index.	Role assignment on the agent identity
C4	**Agent calls arbitrary endpoints**	Agent compute runs in your delegated subnet. Azure Firewall allows only the tool layer and private endpoints.	networkInjections, firewall policy
C5	**Prompt injection triggers a harmful tool call**	List the allowed tools explicitly and require human approval for tools that change data.	allowed_tools, require_approval
C6	**Secrets leak through prompts, logs or code**	Credentials live in project connections backed by Key Vault. Prefer Entra tokens over stored keys.	Project connection
C7	**Agent returns data the user is not entitled to**	On-behalf-of token carries the user's identity, so Microsoft 365 and Fabric apply the user's own permissions and sensitivity labels.	Connection auth type, Purview labels
C8	**Data is sent out to the internet**	Public network access disabled, private endpoints for every dependency, outbound traffic forced through the firewall.	publicNetworkAccess, private endpoints
C9	**Conversation data sits in storage you cannot inspect**	Standard agent setup keeps threads and files in your own resources, encrypted with your key.	Standard setup, customer-managed keys
C10	**Jailbreak, harmful output, injected instructions in documents**	Guardrail policy on the model deployment: content filters, Prompt Shields for user and document attacks.	Guardrail (content filter) policy
C11	**Misuse goes unnoticed and cannot be investigated**	Trace every run and tool call. Defender raises AI threat alerts. Purview applies data-loss rules.	Diagnostic settings, Defender, Purview

SAMPLE CONFIGURATION

Comments marked C1, C2 and so on show which safeguard each line implements. These samples illustrate the safeguards and are not ready-to-deploy templates.

Foundry resource: no keys, no public access (Bicep)

```
resource foundry 'Microsoft.CognitiveServices/accounts@2025-06-01' = {
  name: 'fdry-agents-prod'
  location: 'centralindia'
  kind: 'AIServices'
  identity: { type: 'SystemAssigned' }
  properties: {
    allowProjectManagement: true
    disableLocalAuth: true              // C1: Entra ID only, API keys stop working
    publicNetworkAccess: 'Disabled'     // C8: private endpoint only
    networkAcls: { defaultAction: 'Deny' }
    networkInjections: [                // C4: agent compute runs in your subnet
      {
        scenario: 'agent'
        subnetArmId: agentSubnet.id
        useMicrosoftManagedNetwork: false
      }
    ]
  }
}
```

Agent definition: tool allow-list and approval (Python)

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient
from azure.ai.projects.models import PromptAgentDefinition, MCPTool

project = AIProjectClient(endpoint=PROJECT_ENDPOINT,
                          credential=DefaultAzureCredential())   # no keys

hr_tools = MCPTool(
    server_label="hr_api",
    server_url="https://apim-agents.internal.example.com/hr/mcp",
    allowed_tools=["get_leave_balance", "approve_leave"],   # C5: allow-list
    require_approval="always",                  # C5: a person approves each call
    project_connection_id="hr-api-entra",       # C6: credentials live here
)

agent = project.agents.create_version(
    agent_name="hr-assistant",
    definition=PromptAgentDefinition(
        model="gpt-4.1",
        instructions="You are the HR assistant.",
        tools=[hr_tools],
    ),
)
```

Role assignments: who may call, what the agent may read (Azure CLI)

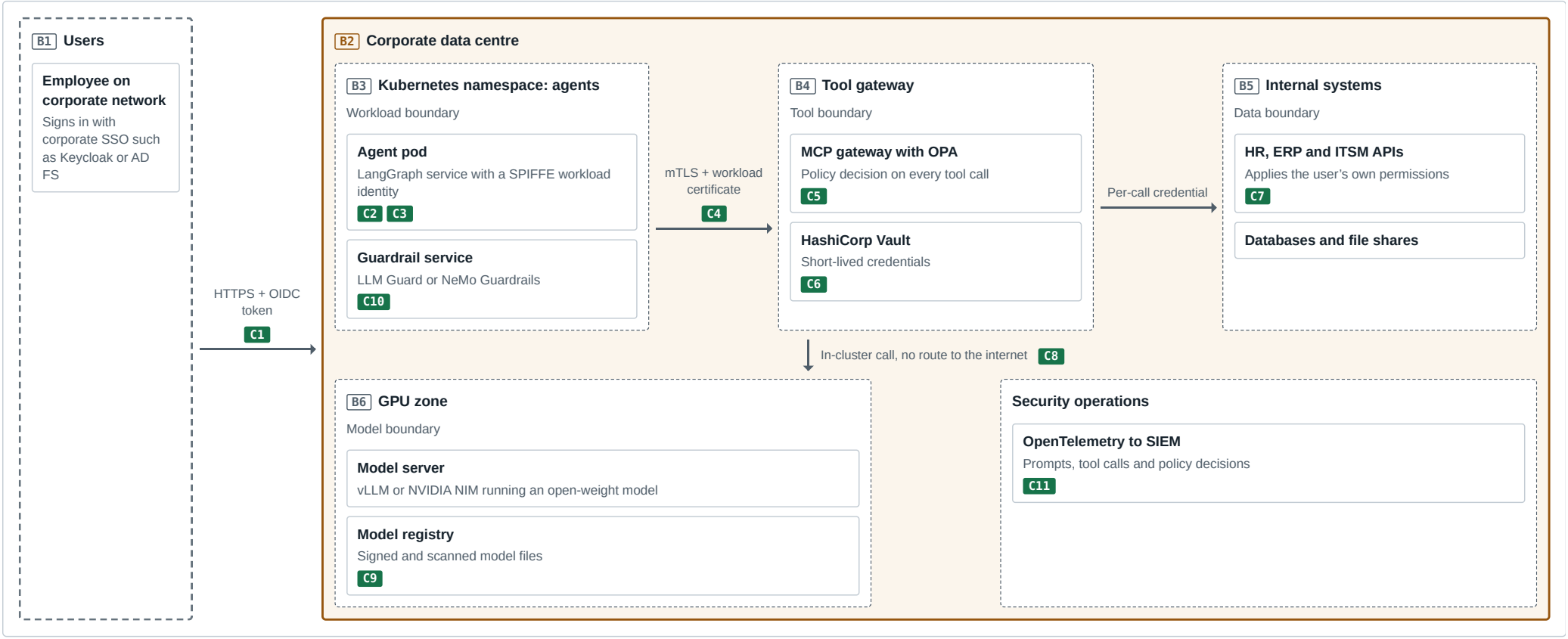
```
# C3: the agent's own identity gets one narrow role on one resource
az role assignment create \
  --assignee-object-id "$AGENT_IDENTITY_OBJECT_ID" \
  --assignee-principal-type ServicePrincipal \
  --role "Search Index Data Reader" \
  --scope "$SEARCH_SERVICE_ID/indexes/hr-policies"

# C1: only the HR portal's identity may call this one agent
# (the portal may show this role under a different display name)
az role assignment create \
  --assignee "$HR_PORTAL_OBJECT_ID" \
  --role "Foundry Agent Consumer" \
  --scope "$FOUNDRY_PROJECT_ID/agents/hr-assistant"
```

Self-hosted inside the organisation’s network

> Here you own the whole building. Nothing leaves your premises, which is the strongest answer to data-residency questions. The cost is that you are also the landlord: every lock, guard and camera that a cloud provider would supply is now yours to build, staff and maintain.

Outside your control · Cloud account · Your data centre · B3 Trust boundary · C5 Control (explained in the table that follows) · Arrow: a request crossing from one zone to the next



WHAT HAPPENS, STEP BY STEP

1. An employee signs in with the company's own single sign-on.
2. Your gateway checks the sign-in and passes the request to the agent, which runs in a locked-down container.
3. The agent sends the question to an AI model running on your own servers. That model has no internet connection.
4. When the model asks for a tool, your policy engine checks who is asking, which tool and with what values.
5. A password vault issues a credential that lasts minutes, and the tool is called as that employee.
6. Your own filter screens the answer. All activity goes to your security operations team.

THE BOUNDARIES IN PLAIN WORDS

BOUNDARY	WHAT IT IS
B1 Network edge	Where users meet the system. Company sign-on is required.
B2 Data centre	Your own premises. Nothing in this design leaves it.
B3 Agent workload	The agent runs in a restricted container that can talk to only two places: the model and the tool gateway.
B4 Tool gateway	Your security desk. A written policy decides every tool request, and a vault hands out short-lived credentials.
B5 Internal systems	HR, finance and IT systems. They apply the employee's own permissions.
B6 GPU zone	The servers that run the AI model. Model files are verified before use, because you are responsible for their integrity.

TECHNICAL SUMMARY

Agent, model and data all sit in your data centre, so nothing leaves the premises. The trade-off is that every control a cloud platform gives you for free, including session isolation, the token vault, guardrails and the integrity of the model itself, is now yours to build and run.

RISKS AND THE SAFEGUARDS THAT ADDRESS THEM

TAG	RISK	SAFEGUARD	WHERE IT IS SET
C1	**Unauthenticated or unauthorised user reaches the agent**	API gateway validates the SSO token: issuer, audience and group membership.	Ingress gateway, agent inbound_auth
C2	**Compromised agent container attacks the node or its neighbours**	Non-root, read-only filesystem, all capabilities dropped, sandboxed runtime such as gVisor or Kata, image pinned by digest.	Pod securityContext, runtimeClassName
C3	**Agents share credentials, so one compromise exposes all**	One service account per agent, issued a short-lived SPIFFE certificate that other services verify.	SPIRE or service mesh
C4	**Agent reaches internal systems directly**	Default-deny network policy. The pod can reach only the model server and the tool gateway, over mutual TLS.	NetworkPolicy, mesh authorisation policy
C5	**Prompt injection triggers a harmful tool call**	OPA policy checks agent identity, user, tool and arguments on every call. Changes to data need a recorded human approval.	Rego policy at the MCP gateway
C6	**Static service passwords in config files**	Vault issues a credential per call that expires in minutes. The model never sees it.	Vault dynamic secrets
C7	**Agent returns data the user is not entitled to**	The gateway passes the user's identity downstream, and the system of record applies its own permissions.	Token exchange, database row-level security
C8	**Model or agent sends data out**	GPU and agent namespaces have no internet egress. Outbound proxy allows named destinations only.	NetworkPolicy, egress proxy
C9	**Tampered or backdoored model file**	Download once into an internal registry, verify signature and hash, scan, and use the safetensors format.	Model registry, admission policy
C10	**Jailbreak, personal data in answers, harmful output**	Self-run guardrail service screens input and output. Step, tool-call and time limits stop runaway loops.	Guardrail service, agent limits
C11	**Misuse goes unnoticed and cannot be investigated**	Send traces for every step to the SIEM. Alert on denied tool calls and unusual volumes.	OpenTelemetry collector, SIEM rules

SAMPLE CONFIGURATION

Comments marked C1, C2 and so on show which safeguard each line implements. These samples illustrate the safeguards and are not ready-to-deploy templates.

Agent workload: locked-down pod and default-deny egress (Kubernetes)

```
apiVersion: apps/v1
kind: Deployment                  # fragment: selectors and labels omitted
metadata: { name: hr-assistant, namespace: agents }
spec:
  template:
    spec:
      serviceAccountName: hr-assistant          # C3: becomes the SPIFFE ID
      automountServiceAccountToken: false
      runtimeClassName: gvisor                  # C2: sandboxed runtime
      containers:
        - name: agent
          image: registry.corp.example.com/agents/hr-assistant@sha256:9f2c...
          securityContext:                      # C2
            runAsNonRoot: true
            readOnlyRootFilesystem: true
            allowPrivilegeEscalation: false
            capabilities: { drop: [ALL] }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy                             # C4, C8: two exits, nothing else
metadata: { name: hr-assistant-egress, namespace: agents }
spec:
  podSelector: { matchLabels: { app: hr-assistant } }
  policyTypes: [Egress]
  egress:                                       # add a rule for cluster DNS
    - to: [{ namespaceSelector: { matchLabels: { name: model-serving } } }]
      ports: [{ port: 8000 }]
    - to: [{ namespaceSelector: { matchLabels: { name: tool-gateway } } }]
      ports: [{ port: 8443 }]
```

Tool policy at the gateway (OPA Rego)

```
package agents.tools
import rego.v1

default allow := false            # C5: anything not listed is denied

hr_agent := "spiffe://corp.example.com/ns/agents/sa/hr-assistant"

# Read: any employee, own record only
allow if {
    input.agent == hr_agent
    input.tool == "hr.get_leave_balance"
    input.args.employee_id == input.user.employee_id
}

# Write: HR staff, small amounts, and a person has approved it
allow if {
    input.agent == hr_agent
    input.tool == "hr.approve_leave"
    "hr-staff" in input.user.groups
    input.args.days <= 5
    input.approval.status == "approved"
}
```

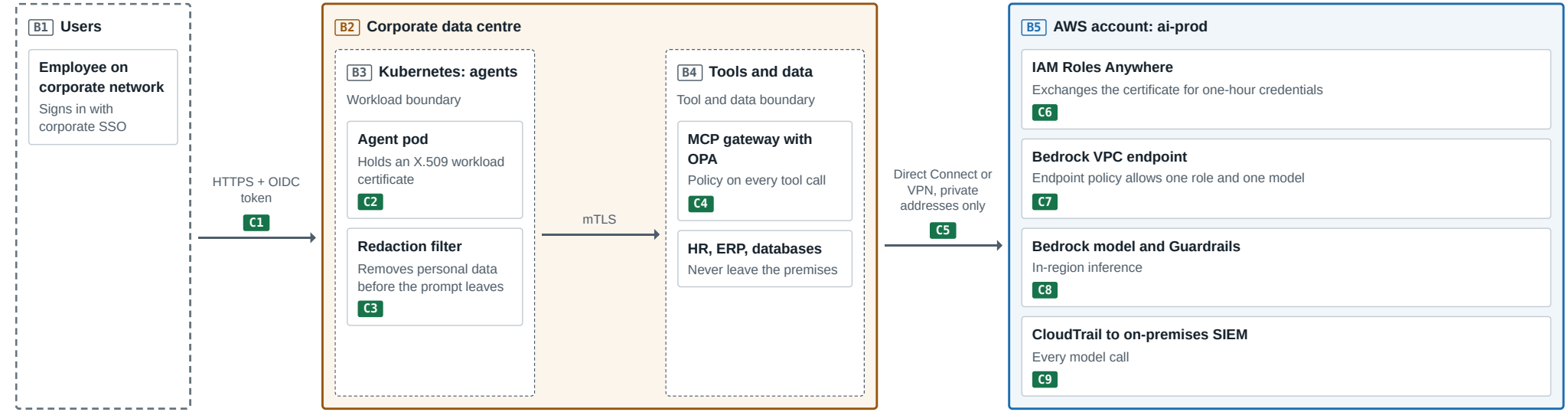
Agent service configuration (your own application config)

```
agent:
  name: hr-assistant
  model:
    endpoint: https://vllm.model-serving.svc:8000/v1    # C8: in-cluster only
    name: llama-3.3-70b-instruct
  inbound_auth:                                          # C1
    issuer: https://sso.corp.example.com/realms/corp
    audience: hr-assistant
    required_groups: [employees]
  tools:
    gateway: https://mcp.tool-gateway.svc:8443           # C4: the only path to tools
    allow: [hr.get_leave_balance, hr.approve_leave]
    require_human_approval: [hr.approve_leave]           # C5
    credentials: vault://agents/hr-assistant             # C6: fetched per call
  guardrails:                                            # C10
    input: [prompt_injection, pii_redact]
    output: [pii_redact, secrets]
  limits: { max_steps: 12, max_tool_calls: 20, timeout_s: 120 }
  telemetry:
    otlp_endpoint: https://otel.soc.corp.example.com:4317   # C11
```

Hybrid with AWS: agent and data on-premises, model in Bedrock

> Your records stay in your own building, and you phone an outside expert (the AI model in AWS) for advice over a private line. The expert never visits your building and never touches your systems. What you must control is the private line, the key used to make the call, and what you say during it.

Outside your control · Cloud account · Your data centre · B3 Trust boundary · C5 Control (explained in the table that follows) · Arrow: a request crossing from one zone to the next



WHAT HAPPENS, STEP BY STEP

1. An employee asks the agent, which runs in your data centre.
2. Before anything leaves, a filter removes personal details from the text.
3. The agent proves its identity to AWS with a certificate from your own certificate authority and receives a pass that lasts one hour.
4. The cleaned-up question travels to the AI model over a private link, not the internet.
5. The model’s reply comes back. If it asks for a tool, your on-premises policy engine decides, not the cloud.
6. Tools and data are used locally. AWS logs each model call and sends the logs to your security team.

THE BOUNDARIES IN PLAIN WORDS

BOUNDARY	WHAT IT IS
B1 Network edge	Where users meet the system. Company sign-on is required.
B2 Data centre	Your premises. The agent, the tools and all business data stay here.
B3 Agent workload	The agent in a restricted container, plus the filter that cleans text before it leaves.
B4 Tools and data	Tool rules are enforced here, on your side, so a reply from the cloud can never act by itself.
B5 AWS account	A small AWS footprint used for one thing only: calling the AI model through a private doorway.

TECHNICAL SUMMARY

The agent, its tools and the business data stay in your data centre. Only the prompt travels to Bedrock, over a private line. The new risks are the line itself, the AWS credential stored on-premises, and what the prompt contains when it leaves. The reverse pattern, with the agent in AgentCore calling tools on-premises, is shown for Azure in the next tab and works the same way on AWS.

RISKS AND THE SAFEGUARDS THAT ADDRESS THEM

TAG	RISK	SAFEGUARD	WHERE IT IS SET
C1	**Unauthenticated user reaches the agent**	API gateway validates the SSO token and group membership.	Ingress gateway
C2	**Compromised agent container**	Same hardening as the self-hosted design: non-root, sandboxed, default-deny network policy with one extra exit to the Bedrock endpoint.	Pod security, NetworkPolicy
C3	**Regulated data leaves the premises inside a prompt**	Redact or tokenise personal data on-premises before the call. Send the minimum context needed.	Redaction step in agent code
C4	**Prompt injection triggers a harmful tool call**	Tool policy is enforced on-premises, so a response from the cloud model can never act on its own.	OPA at the MCP gateway
C5	**Traffic intercepted or sent over the public internet**	Private connectivity with TLS on top. Call the VPC endpoint address, never the public Bedrock endpoint. Block the public endpoint at the proxy.	Direct Connect, endpoint_url
C6	**Long-lived AWS access key sitting on an on-premises server**	No access keys. The workload certificate from your own CA is exchanged for credentials that expire in an hour.	IAM Roles Anywhere trust anchor and role
C7	**Stolen credentials used from somewhere else**	The role works only through your VPC endpoint and only in one region. The endpoint policy names the role and the model.	aws:SourceVpce condition, endpoint policy
C8	**Jailbreak or harmful output; data processed outside the country**	Guardrail applied on every call. Use an in-region model ID so inference is not routed to other regions.	guardrailConfig, model ID choice
C9	**Cloud usage invisible to the on-premises security team**	CloudTrail records each model call with the certificate identity. Forward to the SIEM and correlate with agent traces.	CloudTrail, sourceIdentity

SAMPLE CONFIGURATION

Comments marked C1, C2 and so on show which safeguard each line implements. These samples illustrate the safeguards and are not ready-to-deploy templates.

Role trust: on-premises certificate instead of access keys (IAM)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": { "Service": "rolesanywhere.amazonaws.com" },
    "Action": ["sts:AssumeRole", "sts:TagSession", "sts:SetSourceIdentity"],
    "Condition": {
      "ArnEquals": {
        "aws:SourceArn": "arn:aws:rolesanywhere:ap-south-1:111122223333:trust-anchor/TA_ID"
      },
      "StringEquals": {
        "aws:PrincipalTag/x509Subject/CN": "hr-assistant.agents.corp.example.com"
      }
    }
  }]
}
```

Role permissions: one model, private endpoint only, one region (IAM)

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Sid": "ModelThroughPrivateEndpointOnly",
    "Effect": "Allow",
    "Action": ["bedrock:InvokeModel", "bedrock:ApplyGuardrail"],
    "Resource": [
      "arn:aws:bedrock:ap-south-1::foundation-model/MODEL_ID",
      "arn:aws:bedrock:ap-south-1:111122223333:guardrail/gr-hr01"
    ],
    "Condition": {
      "StringEquals": {
        "aws:SourceVpce": "vpce-0abc123def456",
        "aws:RequestedRegion": "ap-south-1"
      }
    }
  }]
}
```

On-premises agent: credentials and model call (AWS config + Python)

```
# ~/.aws/config inside the agent pod (C6): no access keys anywhere
[profile bedrock-agent]
region = ap-south-1
credential_process = aws_signing_helper credential-process \
  --certificate /var/run/svid/cert.pem --private-key /var/run/svid/key.pem \
  --trust-anchor-arn arn:aws:rolesanywhere:ap-south-1:111122223333:trust-anchor/TA_ID \
  --profile-arn arn:aws:rolesanywhere:ap-south-1:111122223333:profile/PROFILE_ID \
  --role-arn arn:aws:iam::111122223333:role/onprem-hr-agent

# agent/model.py
import boto3

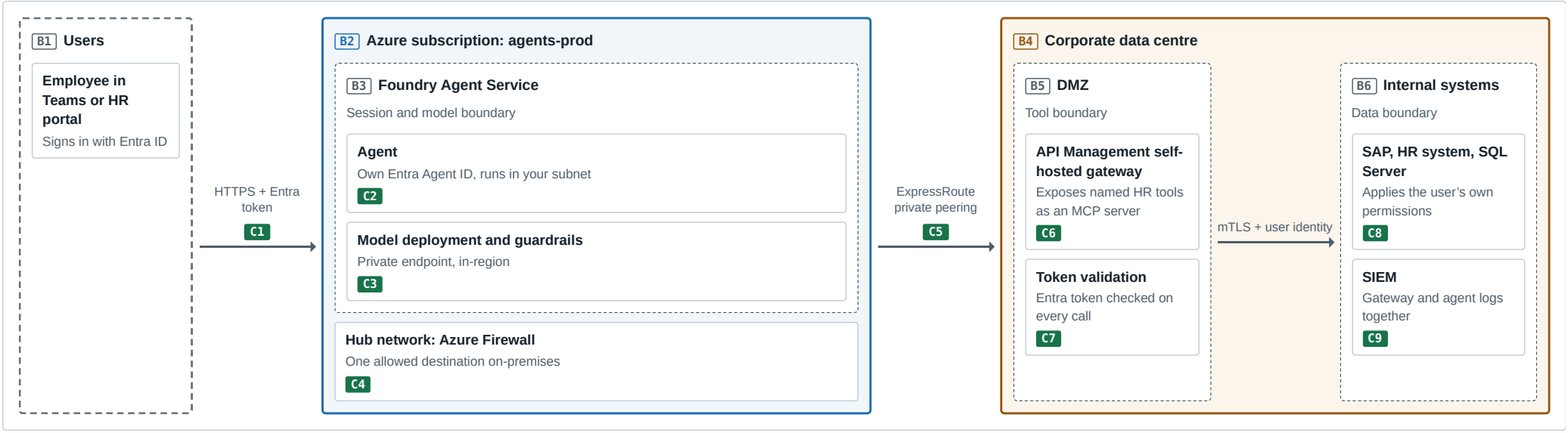
session = boto3.Session(profile_name="bedrock-agent")
bedrock = session.client(
    "bedrock-runtime",
    endpoint_url="https://vpce-0abc123def456.bedrock-runtime.ap-south-1.vpce.amazonaws.com",
)                                           # C5: private path only

def ask_model(messages):
    clean = redact_pii(messages)              # C3: runs before anything leaves
    return bedrock.converse(
        modelId=MODEL_ID,
        messages=clean,
        guardrailConfig={"guardrailIdentifier": "gr-hr01",
                         "guardrailVersion": "3"},          # C8
    )
```

Hybrid with Azure: agent in Foundry, tools and data on-premises

> This is the reverse of the previous design. The agent works from Microsoft's office and needs to consult records kept in your building. You give it one guarded door, a visitor desk that checks its pass every time, and a short list of things it may ask for. It never gets the run of the building.

Outside your control · Cloud account · Your data centre · B3 Trust boundary · C5 Control (explained in the table that follows) · Arrow: a request crossing from one zone to the next



WHAT HAPPENS, STEP BY STEP

1. An employee signs in and asks the agent, which runs in Azure.
2. The AI model decides it needs information from an on-premises system.
3. The request leaves Azure through a firewall that allows exactly one destination in your data centre.
4. It travels over a private link to a gateway in your network’s buffer zone (DMZ).
5. The gateway checks the pass on every call and offers only a short list of named tools.
6. Your internal system answers using the employee’s own permissions. Sensitive fields are removed before the reply goes back.

THE BOUNDARIES IN PLAIN WORDS

BOUNDARY	WHAT IT IS
B1 Internet edge	Where users meet the system. Microsoft work accounts only.
B2 Azure subscription	Your private space in Azure, where the agent and model run.
B3 Agent and model	The agent with its own identity, and the AI model with its content filters.
B4 Data centre	Your premises, connected to Azure by a private link.
B5 DMZ	The buffer zone. One gateway receives agent requests and checks each one.
B6 Internal systems	The systems of record. They never accept a request directly from the cloud.

TECHNICAL SUMMARY

The agent and model run in Azure, and the systems of record stay in your data centre. The main risk is that a cloud agent becomes a way into the internal network. The answer is one narrow, authenticated path: a single gateway on-premises that exposes named tools only and checks an Entra token on every call. The reverse pattern, with the agent on-premises calling a cloud model, is shown for AWS in the previous tab and works the same way with Azure private endpoints.

RISKS AND THE SAFEGUARDS THAT ADDRESS THEM

TAG	RISK	SAFEGUARD	WHERE IT IS SET
C1	**Anonymous caller or leaked API key invokes the agent**	API keys off, Entra ID sign-in only, role scoped to the one agent, Conditional Access for callers.	disableLocalAuth, Azure RBAC
C2	**Agent identity is compromised or misused**	Dedicated Entra Agent ID with no standing rights in Azure beyond its job. Disabling it cuts on-premises access at once.	Entra Agent ID, access reviews
C3	**Internal data is processed or kept outside policy**	Model deployed in an approved region with a regional deployment type. Guardrail policy applied. Threads stored in your own resources.	Deployment type, standard agent setup
C4	**Cloud agent used as a stepping stone into the internal network**	Agent subnet traffic is forced through the firewall, which allows one on-premises address on port 443 and nothing else.	Route table, firewall policy
C5	**Traffic intercepted in transit**	Private peering, no public exposure of the on-premises gateway, TLS end to end.	ExpressRoute, private DNS
C6	**Agent can call any internal API it discovers**	The gateway publishes a short list of tools, not network access. Anything not published does not exist for the agent.	Gateway API definitions
C7	**Forged or replayed token**	Validate tenant, audience, calling application and scope on every request. Tokens are short-lived and bound to one audience.	validate-azure-ad-token policy
C8	**Agent returns data the user is not entitled to**	The user’s identity is passed through, and the system of record applies its own permissions. The gateway strips fields the model does not need.	On-behalf-of flow, outbound policy
C9	**Two halves of an incident are logged in two places**	Send Foundry traces and on-premises gateway logs to one SIEM, joined by a correlation ID.	Diagnostic settings, gateway logging

SAMPLE CONFIGURATION

Comments marked C1, C2 and so on show which safeguard each line implements. These samples illustrate the safeguards and are not ready-to-deploy templates.

Agent definition: on-premises tools through one private URL (Python)

```
from azure.ai.projects.models import PromptAgentDefinition, MCPTool

onprem_hr = MCPTool(
    server_label="onprem_hr",
    # Resolves through private DNS to the gateway in the DMZ (C5)
    server_url="https://hr-tools.dmz.corp.example.com/mcp",
    allowed_tools=["get_leave_balance", "approve_leave"],   # C6
    require_approval="always",
    project_connection_id="onprem-hr-entra",   # Entra token, not a stored key
)

agent = project.agents.create_version(
    agent_name="hr-assistant",
    definition=PromptAgentDefinition(
        model="gpt-4.1",                        # C3: regional deployment
        instructions="You are the HR assistant.",
        tools=[onprem_hr],
    ),
)
```

Firewall: one destination on-premises (Azure CLI)

```
# C4: the agent subnet may reach exactly one address, on one port
az network firewall policy rule-collection-group collection add-filter-collection \
  --resource-group rg-hub --policy-name fw-hub \
  --rule-collection-group-name agents \
  --name allow-onprem-tool-gateway --collection-priority 200 --action Allow \
  --rule-type NetworkRule --rule-name foundry-to-dmz-gateway \
  --source-addresses 10.20.4.0/24 \
  --destination-addresses 10.250.10.15 \
  --destination-ports 443 --ip-protocols TCP
```

On-premises gateway: check the token on every call (API Management policy)

```
<policies>
  <inbound>
    <!-- C7: reject anything that is not our tenant, our API, our agent.
         Which application ID appears in the token depends on the
         Agent ID flow in use; confirm it in a test token first. -->
    <validate-azure-ad-token tenant-id="{{tenant-id}}">
      <client-application-ids>
        <application-id>{{hr-agent-app-id}}</application-id>
      </client-application-ids>
      <audiences>
        <audience>api://hr-onprem-api</audience>
      </audiences>
      <required-claims>
        <claim name="scp" match="any">
          <value>Leave.Read</value>
          <value>Leave.Approve</value>
        </claim>
      </required-claims>
    </validate-azure-ad-token>
    <rate-limit-by-key calls="30" renewal-period="60"
                       counter-key="@(context.Request.IpAddress)" />
  </inbound>
  <outbound>
    <!-- C8: remove fields the model never needs, e.g. salary, bank details -->
  </outbound>
</policies>
```

Notes for use

The sample configuration in this document is written to explain the safeguards. Property names change between service versions and some of the cloud services shown are in preview, so check each sample against current AWS and Microsoft documentation before use. Account numbers, host names and resource names are placeholders.

The two hybrid designs show opposite patterns on purpose. Either pattern can be built on either cloud.